29.3 技能安装与管理

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~420k tokens，输出 ~6k tokens（本节）

技能声明了外部依赖（如 `gh` CLI、 `curl` ），但这些依赖不一定已安装在用户机器上。OpenClaw 的技能安装系统让用户（或 Agent）能一键安装缺失的依赖，同时提供技能状态审计能力。

29.3.1 技能安装流程

安装入口

`skills-install.ts` 实现了完整的技能依赖安装流程。入口是 `installSkillDependency` （由 CLI 或 Agent 调用），核心逻辑如下：

```
// src/agents/skills-install.ts（简化概念流程）

async function installSkillDependency(request: SkillInstallRequest): Promise<SkillInsta
  // 1. 加载技能条目
  const entries = loadWorkspaceSkillEntries(request.workspaceDir, { config: request.con
  const entry = entries.find(e => e.skill.name === request.skillName);

  // 2. 查找指定的安装规格
  const spec = findInstallSpec(entry, request.installId);

  // 3. 安全扫描（安装前检查技能代码）
  const warnings = await collectSkillInstallScanWarnings(entry);

  // 4. 根据安装类型执行
  if (spec.kind === "download") {
    return await installDownloadSpec({ entry, spec, timeoutMs });
  }

  // 5. 构建并执行安装命令
  const { argv, error } = buildInstallCommand(spec, prefs);
  const result = await runCommandWithTimeout(argv, { timeoutMs });

  return { ok: result.code === 0, message, stdout, stderr, warnings };
}
```

安装命令构建

`buildInstallCommand` 根据安装类型和用户偏好生成具体命令：

```typescript
// src/agents/skills-install.ts（简化）

function buildInstallCommand(spec: SkillInstallSpec, prefs: SkillsInstallPreferences) {
  switch (spec.kind) {
    case "brew":
      return { argv: ["brew", "install", spec.formula] };
    case "node":
      return { argv: buildNodeInstallCommand(spec.package, prefs) };
    case "go":
      return { argv: ["go", "install", spec.module] };
    case "uv":
      return { argv: ["uv", "tool", "install", spec.package] };
    case "download":
      return { argv: null, error: "download handled separately" };
  }
}

function buildNodeInstallCommand(packageName: string, prefs): string[] {
  switch (prefs.nodeManager) {
    case "pnpm":  return ["pnpm", "add", "-g", packageName];
    case "yarn":  return ["yarn", "global", "add", packageName];
    case "bun":   return ["bun", "add", "-g", packageName];
    default:      return ["npm", "install", "-g", packageName];
  }
}
```

Node.js 包管理器的选择通过配置决定：

```typescript
// src/agents/skills.ts

export function resolveSkillsInstallPreferences(config?): SkillsInstallPreferences {
  const raw = config?.skills?.install;
  return {
    preferBrew: raw?.preferBrew ?? true,    // 默认优先使用 Homebrew
    nodeManager: raw?.nodeManager ?? "npm", // 默认使用 npm
  };
}
```

Download 安装器

`download` 类型的安装器处理直接下载二进制文件或归档包的场景：

```typescript
// src/agents/skills-install.ts（简化）

async function installDownloadSpec(params): Promise<SkillInstallResult> {
  const { spec, entry, timeoutMs } = params;
  const targetDir = resolveDownloadTargetDir(entry, spec);

  // 1. 下载文件
  await downloadFile(spec.url, archivePath, timeoutMs);

  // 2. 如果是归档文件，解压
  const archiveType = resolveArchiveType(spec, filename);
  if (archiveType && spec.extract !== false) {
    await extractArchive({
      archivePath, archiveType, targetDir,
      stripComponents: spec.stripComponents,
      timeoutMs,
    });
  }

  // 3. 设置可执行权限
  if (spec.bins?.length) {
    for (const bin of spec.bins) {
      const binPath = path.join(targetDir, bin);
      await fs.promises.chmod(binPath, 0o755);
    }
  }
}
```

支持的归档格式： `tar.gz` 、 `tar.bz2` 、 `zip` 。 `stripComponents` 参数用于去除归档内的顶层目录（类似 `tar --strip-components` ）。

安全扫描

安装前，系统会对技能目录执行安全扫描：

```
// src/agents/skills-install.ts（简化）

async function collectSkillInstallScanWarnings(entry: SkillEntry): Promise<string[]> {
  const summary = await scanDirectoryWithSummary(entry.skill.baseDir);
  if (summary.critical > 0) {
    warnings.push(`WARNING: Skill "${name}" contains dangerous code patterns: ${details
  } else if (summary.warn > 0) {
    warnings.push(`Skill "${name}" has ${summary.warn} suspicious code pattern(s).`);
  }
  return warnings;
}
```

> **衍生解释**：安全扫描检查技能目录中是否包含可疑的代码模式——例如从未知来源下载并执行文件、访问敏感系统路径等。这对于从 ClawHub 安装的第三方技能尤其重要，因为技能内容最终会被注入到 Agent 的提示词中，恶意技能可能试图操纵 Agent 行为。

29.3.2 技能状态管理

状态报告

`skills-status.ts` 提供完整的技能状态审计功能。 `buildSkillStatusReport` 为每个技能生成详细的状态报告：

```typescript
// src/agents/skills-status.ts（简化）

export type SkillStatusEntry = {
  name: string;
  description: string;
  source: string;              // "openclaw-bundled" | "openclaw-managed" 等
  bundled: boolean;
  eligible: boolean;           // 是否通过资格检查
  disabled: boolean;           // 是否被配置禁用
  blockedByAllowlist: boolean; // 是否被白名单阻止
  requirements: {              // 声明的依赖
    bins: string[];
    anyBins: string[];
    env: string[];
    config: string[];
    os: string[];
  };
  missing: {                   // 缺失的依赖
    bins: string[];
    anyBins: string[];
    env: string[];
    config: string[];
    os: string[];
  };
  install: SkillInstallOption[];  // 可用的安装选项
};
```

`missing` 字段是关键——它精确列出了阻止技能激活的缺失依赖，用户可以据此决定是否安装。

安装选项选择

当一个技能提供多种安装方式时， `selectPreferredInstallSpec` 按照用户偏好排序：

```
// src/agents/skills-status.ts（简化）

function selectPreferredInstallSpec(install: SkillInstallSpec[], prefs) {
  // 1. 如果用户偏好 Homebrew 且 brew 已安装 → 选 brew
  if (prefs.preferBrew && hasBinary("brew") && brewSpec) return brewSpec;
  // 2. uv（Python）→ node → go → brew（非偏好时）→ download
  if (uvSpec) return uvSpec;
  if (nodeSpec) return nodeSpec;
  if (goSpec) return goSpec;
  if (brewSpec) return brewSpec;
  return downloadSpec ?? install[0];
}
```

29.3.3 技能 CLI 命令

OpenClaw CLI 提供了完整的技能管理命令集。虽然具体 CLI 实现（`src/cli/skills-cli.ts`）涉及终端 UI 渲染等细节，核心功能包括：

常用命令

```
# 列出所有技能及其状态
openclaw skills list

# 查看特定技能详情（依赖、安装选项）
openclaw skills info weather

# 安装技能依赖
openclaw skills install github brew

# 刷新技能列表
openclaw skills refresh
```

技能同步

对于沙箱（Sandbox）环境，技能需要从主工作区复制到沙箱目录。`syncSkillsToWorkspace` 处理这个场景：

```ts
// src/agents/skills/workspace.ts（简化）

export async function syncSkillsToWorkspace(params: {
  sourceWorkspaceDir: string;
  targetWorkspaceDir: string;
  config?: OpenClawConfig;
}) {
  // 1. 从源目录加载所有技能
  const entries = loadSkillEntries(sourceWorkspaceDir, { config });

  // 2. 清空目标 skills/ 目录
  await fs.promises.rm(targetSkillsDir, { recursive: true, force: true });
  await fs.promises.mkdir(targetSkillsDir, { recursive: true });

  // 3. 逐个复制技能目录
  for (const entry of entries) {
    await fs.promises.cp(entry.skill.baseDir, destDir, { recursive: true });
  }
}
```

同步通过 `serializeByKey` 序列化执行，确保同一目标目录不会并发写入。

环境变量覆盖

技能可以通过配置声明环境变量覆盖，在 Agent 执行工具时注入：

```typescript
// src/agents/skills/env-overrides.ts

export function applySkillEnvOverrides(params: {
  config: OpenClawConfig;
  skills: Array<{ name: string; primaryEnv?: string }>;
}): Record<string, string> {
  const overrides: Record<string, string> = {};
  for (const skill of params.skills) {
    const skillConfig = resolveSkillConfig(params.config, skill.name);
    // 从技能配置中提取 env 覆盖
    if (skillConfig?.env) {
      Object.assign(overrides, skillConfig.env);
    }
    // apiKey → primaryEnv 映射
    if (skillConfig?.apiKey && skill.primaryEnv) {
      overrides[skill.primaryEnv] = skillConfig.apiKey;
    }
  }
  return overrides;
}
```

这允许用户在 `settings.json` 中配置技能的 API Key，而不需要设置全局环境变量：

```json
{
  "skills": {
    "entries": {
      "discord": {
        "apiKey": "your-discord-token",
        "env": {
          "DISCORD_GUILD_ID": "123456789"
        }
      }
    }
  }
}
```

配置的 `apiKey` 会自动映射到技能声明的 `primaryEnv`（对 Discord 来说是 `DISCORD_TOKEN`），使技能通过环境变量检查。

本节小结

1. **安装流程**支持五种安装器（brew/node/go/uv/download），根据用户偏好自动选择最合适的包管理器，安装前执行安全扫描。
2. **状态报告**精确列出每个技能的依赖、缺失项和可用安装选项，帮助用户了解为什么某些技能未激活。
3. **CLI 命令**提供 `list` 、 `info` 、 `install` 、 `refresh` 等操作，覆盖技能生命周期的各个阶段。
4. **沙箱同步**将主工作区的技能复制到沙箱环境，确保隔离执行时技能依然可用。
5. **环境变量覆盖**让用户通过配置文件管理技能的 API Key 和环境变量，无需修改系统环境。